1 Introduction and Lab Objectives

In this two-hour lab session, we build a **simple feedforward neural network** (also called a Multi-Layer Perceptron, MLP) from scratch using only NumPy. The key constraint is that we perform **forward prediction only** (no backpropagation or training).

1.1 What You Will Learn

1. How a neural network is structured as a sequence of **layers**.
2. What **weights**, **biases**, and **activation functions** are.
3. How data flows through a network (**forward pass**).
4. How to measure prediction quality using **loss functions** (MSE and MAE).
5. Why **weight initialization** matters.
6. How to generate simple **synthetic regression data**.

1.2 Why Forward Pass Only?

Before learning how to *train* a network (which involves backpropagation and gradient descent), it is essential to understand how a network *computes* its output. Think of it like understanding how a car engine works before learning to drive.

2 Section 1: Importing NumPy

```python
import numpy as np
```

2.1 Explanation

NumPy is the fundamental library for numerical computing in Python. In the context of neural networks, we use it for:

- **Matrix multiplication** – the core operation in every neural network layer.
- **Element-wise operations** – for activation functions and loss computation.
- **Random number generation** – for initializing weights.
- **Broadcasting** – for adding biases efficiently across batches.

2.2 Key Teaching Point

Neural networks are fundamentally **linear algebra machines**. Every forward pass is a sequence of matrix multiplications and nonlinear transformations. NumPy lets us express these operations concisely and efficiently.

3 Section 2: Fixing the Random Seed

```
def set_seed(seed=42):
    np.random.seed(seed)
```

3.1 Explanation

Computers generate **pseudo-random** numbers using deterministic algorithms. By fixing the seed, we ensure that:

- Every run produces the **same** random weights and data noise.
- Results are **reproducible** – essential for debugging and scientific reporting.

3.2 Key Teaching Point

Reproducibility is a cornerstone of scientific computing. In machine learning research and coursework, always set a random seed so that your results can be verified and compared.

> **Analogy:** Think of the seed as a "recipe number." Given the same recipe number, the random number generator always produces the same sequence of numbers, just like following the same recipe always produces the same dish.

4 Section 3: Activation Functions

```
def relu(z):
    return np.maximum(0, z)

def tanh(z):
    return np.tanh(z)

def sigmoid(z):
    return 1 / (1 + np.exp(-z))
```

4.1 Explanation

Activation functions introduce **nonlinearity** into the network. Without them, stacking multiple layers would be equivalent to a single linear transformation.

4.2 The Three Activation Functions

4.3 Why Nonlinearity Matters

Consider two linear layers:

$$\mathbf{h} = \mathbf{x}\mathbf{W}_1 + \mathbf{b}_1 \tag{1}$$

$$\mathbf{y} = \mathbf{h}\mathbf{W}_2 + \mathbf{b}_2 \tag{2}$$

Function	Formula	Range	Use Case
ReLU	$f(z) = \max(0, z)$	$[0, +\infty)$	Hidden layers (most common)
Tanh	$f(z) = \tanh(z)$	$(-1, +1)$	Hidden layers (centered output)
Sigmoid	$f(z) = \dfrac{1}{1 + e^{-z}}$	$(0, 1)$	Binary classification output

Table 1: Summary of activation functions used in this lab.

Substituting:

$$\mathbf{y} = \mathbf{x}(\mathbf{W}_1\mathbf{W}_2) + (\mathbf{b}_1\mathbf{W}_2 + \mathbf{b}_2) = \mathbf{x}\mathbf{W}' + \mathbf{b}'$$

This collapses into a **single linear layer**. Activation functions break this collapse and allow the network to learn complex, nonlinear mappings.

4.4 Key Teaching Point

- **ReLU** is the default choice for hidden layers due to its simplicity and effectiveness.
- The output layer activation depends on the task: **None (linear)** for regression, **sigmoid** for binary classification, **softmax** for multi-class.

5 Section 4: The Dense Layer Class

```python
class DenseLayer:
    def __init__(self, in_dim, out_dim, activation=None):
        self.in_dim = in_dim
        self.out_dim = out_dim
        self.activation = activation

        # Xavier initialization
        limit = np.sqrt(6 / (in_dim + out_dim))
        self.W = np.random.uniform(-limit, limit, (in_dim, out_dim))
        self.b = np.zeros(out_dim)

    def forward(self, x):
        z = np.dot(x, self.W) + self.b
        if self.activation is None:
            return z
        else:
            return self.activation(z)
```

5.1 Explanation

This is the **core building block** of our neural network. Each dense (fully connected) layer performs:

$$\mathbf{z} = \mathbf{x}\mathbf{W} + \mathbf{b}, \qquad \mathbf{a} = f(\mathbf{z})$$

where:

- $\mathbf{x} \in \mathbb{R}^{n \times d_{\text{in}}}$ is the input (batch of n samples),
- $\mathbf{W} \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$ is the weight matrix,

- $\mathbf{b} \in \mathbb{R}^{d_{\text{out}}}$ is the bias vector,
- $f(\cdot)$ is the activation function.

5.2 Xavier Initialization

The weights are initialized using **Xavier (Glorot) initialization**:

$$W_{ij} \sim \mathcal{U}\left(-\sqrt{\frac{6}{d_{\text{in}} + d_{\text{out}}}}, \sqrt{\frac{6}{d_{\text{in}} + d_{\text{out}}}}\right)$$

Why not just use random values like $\mathcal{U}(-1, 1)$?

- If weights are too large, activations **explode** (grow to infinity).
- If weights are too small, activations **vanish** (shrink to zero).
- Xavier initialization keeps the **variance of activations** roughly constant across layers.

5.3 Bias Initialization

Biases are initialized to **zero**. This is standard practice because the random weights already break symmetry.

5.4 Key Teaching Point

A single dense layer is just a linear equation $\mathbf{z} = \mathbf{x}\mathbf{W} + \mathbf{b}$ followed by a nonlinear activation. The entire neural network is a **composition** of such layers.

6 Section 5: Loss Functions

```python
def mse_loss(y_pred, y_true):
    diff = y_pred - y_true
    return np.mean(diff ** 2)

def mae_loss(y_pred, y_true):
    diff = y_pred - y_true
    return np.mean(np.abs(diff))
```

6.1 Explanation

Loss functions measure **how wrong** the network's predictions are.

6.1.1 Mean Squared Error (MSE)

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2$$

- Penalizes **large errors** more heavily (quadratic penalty).
- Differentiable everywhere – convenient for gradient-based training.
- Sensitive to outliers.

6.1.2 Mean Absolute Error (MAE)

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|\hat{y}_i - y_i|$$

- Treats all errors **equally** (linear penalty).
- More **robust to outliers** than MSE.
- Not differentiable at zero (but subgradients exist).

6.2 Key Teaching Point

The choice of loss function defines what "good" means for your model. MSE and MAE are the two most common losses for regression tasks. In this lab, since we do not train, the loss simply tells us how far off our random network's predictions are.

7 Section 6: Generating Toy Regression Data

```python
def make_regression_data(n=100):
    X = np.linspace(-np.pi, np.pi, n).reshape(-1, 1)
    noise = 0.05 * np.random.randn(n, 1)
    y = np.sin(X) + noise
    return X, y
```

7.1 Explanation

We generate synthetic data following:

$$y = \sin(x) + \epsilon, \qquad x \in [-\pi, \pi], \quad \epsilon \sim \mathcal{N}(0, 0.05^2)$$

- $\sin(x)$ is a smooth, nonlinear function – a good test for neural networks.
- The small Gaussian noise ϵ simulates real-world measurement imperfection.
- `reshape(-1, 1)` converts the 1D array into a column vector (shape 100×1), which is the expected input format for our network.

7.2 Key Teaching Point

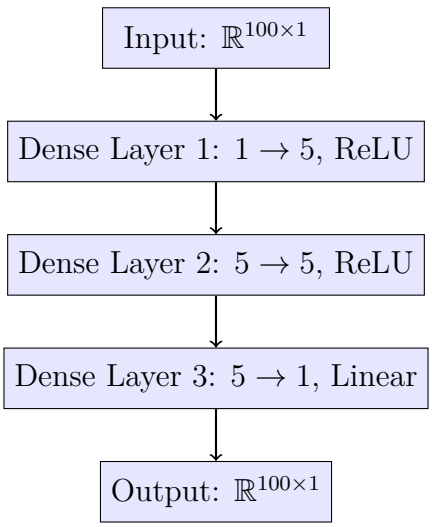
Using synthetic data lets us **know the true underlying function**, so we can clearly see how well (or poorly) the network approximates it. In real applications, we never know the true function.

8 Section 7: Main Demonstration – Putting It All Together

```python
def main():
    set_seed(123)
    X, y = make_regression_data()

    # Create layers
    layer1 = DenseLayer(1, 5, activation=relu)
    layer2 = DenseLayer(5, 5, activation=relu)
    layer3 = DenseLayer(5, 1, activation=None)

    # Forward pass
    h1 = layer1.forward(X)
    h2 = layer2.forward(h1)
    y_pred = layer3.forward(h2)

    # Compute losses
    mse = mse_loss(y_pred, y)
    mae = mae_loss(y_pred, y)

    print("Predicted y shape:", y_pred.shape)
    print(f"MSE: {mse:.5f}, MAE: {mae:.5f}")
```

8.1 Network Architecture

The network has the following structure:



8.2 Step-by-Step Data Flow

1. **Input X**: shape $(100, 1)$ – 100 data points, each with 1 feature.
2. **Layer 1**: $\mathbf{h}_1 = \text{ReLU}(\mathbf{X}\mathbf{W}_1 + \mathbf{b}_1)$, shape $(100, 5)$.
3. **Layer 2**: $\mathbf{h}_2 = \text{ReLU}(\mathbf{h}_1\mathbf{W}_2 + \mathbf{b}_2)$, shape $(100, 5)$.
4. **Layer 3**: $\hat{\mathbf{y}} = \mathbf{h}_2\mathbf{W}_3 + \mathbf{b}_3$, shape $(100, 1)$.
5. **Loss**: Compare $\hat{\mathbf{y}}$ with true $\mathbf{y}$ using MSE and MAE.

8.3 Why Are the Losses High?

Since we use **random weights** and perform **no training**, the network output is essentially a random nonlinear transformation of the input. The losses will be high, which is expected. This demonstrates that:

- A network with random weights is **not useful** for prediction.
- **Training** (adjusting weights via backpropagation) is what makes neural networks powerful.

9 Section 8: Running the Code

```python
if __name__ == "__main__":
    main()
```

9.1 Explanation

This is a Python idiom that ensures `main()` runs only when the script is executed directly (not when imported as a module). It is considered best practice for organizing Python programs.

10 Summary of Key Concepts

Concept	What to Remember
Dense Layer	$\mathbf{z} = \mathbf{x}\mathbf{W} + \mathbf{b}$, then apply activation
Activation Function	Introduces nonlinearity (ReLU, Tanh, Sigmoid)
Xavier Initialization	Keeps activations stable across layers
Forward Pass	Data flows input $\rightarrow$ hidden layers $\rightarrow$ output
MSE Loss	$\frac{1}{n}\sum(\hat{y} - y)^2$ – penalizes large errors
MAE Loss	$\frac{1}{n}\sum\lvert\hat{y} - y\rvert$ – robust to outliers
Random Seed	Ensures reproducibility
No Training	Random weights $\Rightarrow$ poor predictions

Table 2: Key concepts from this lab session.

11 Discussion Questions for Students

1. What happens if we remove all activation functions? Is the network still useful?
2. Why do we use a **linear** (no activation) output layer for regression?
3. What would happen if we initialized all weights to zero?
4. How would the output change if we used `tanh` instead of `relu` in the hidden layers?
5. Why is the MSE loss much larger than the MAE loss in our experiment?
6. What is the next step needed to make this network actually learn? (Hint: back-propagation)